

Name: K.Likitha

Hall Ticket: 2303A52144

Batch: 41

Task 1: Password Strength & Validation Check

Prompt:

Generate a simple Python user registration program and then improve it by enforcing strong password validation including minimum length of at least 8 characters, requiring a mix of uppercase, lowercase, digits and special characters using regular expressions, and rejecting common or weak passwords, finally output the secure version of the script with proper validation implemented

Code:

```
#Generate a simple Python user registration program and then improve it by enforcing
strong password validation including minimum length of at least 8 characters, requiring a
mix of uppercase, lowercase, digits and special characters using regular expressions, and
rejecting common or weak passwords, finally output the secure version of the script with
proper validation implemented
# Simple User Registration Program
'''def register_user():
    username = input("Enter a username: ")
    password = input("Enter a password: ")
    print(f"User '{username}' registered successfully!")
register_user()'''
# Improved User Registration Program with Strong Password Validation
import re
def is_strong_password(password):
    if len(password) < 8:
        return False
    if not re.search(r'[A-Z]', password):
        return False
    if not re.search(r'[a-z]', password):
        return False
    if not re.search(r'[0-9]', password):
        return False
    if not re.search(r'[@$!%*?&]', password):
        return False
    common_passwords = ['password', '123456', '12345678', 'qwerty', 'abc123']
    if password.lower() in common_passwords:
        return False
    return True
def register_user():
    username = input("Enter a username: ")
    while True:
        password = input("Enter a password: ")
        if is_strong_password(password):
            print(f"User '{username}' registered successfully!")
            break
        else:
            print("Weak password. Please enter a stronger password that is at least 8
characters long, includes uppercase and lowercase letters, digits, and special characters,
and is not a common password.")
```

```
register_user()
```

Output:

```
PS C:\Users\lilac\New folder\fashion-store\server> & C:\Users\lilac\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/lilac/New folder/fashion-store/s
erver/ai3.py"
Enter a username: Likitha_2144
Enter a password: likitha123
Weak password. Please enter a stronger password that is at least 8 characters long, includes uppercase and lowercase letters, digits, and special characters, and
is not a common password.
Enter a password: Lik!th@.123
User 'Likitha_2144' registered successfully!
PS C:\Users\lilac\New folder\fashion-store\server> []
```

Explanation:

This code creates a user registration system that checks if the entered password is strong and secure before accepting it. It prevents weak passwords by enforcing rules like minimum length, character variety, and blocking commonly used passwords.

Task 2: Insecure File Upload Handling

Prompt:

Generate a Python Flask file upload program and improve it by adding security features such as validating allowed file extensions, restricting file size limits, and safely storing uploaded files using secure paths and filenames to prevent malicious uploads

Code:

```
from flask import Flask, request, render_template_string
import os
from werkzeug.utils import secure_filename

app = Flask(__name__)

# Configuration
UPLOAD_FOLDER = 'uploads'
MAX_FILE_SIZE = 16 * 1024 * 1024 # 16 MB
ALLOWED_EXTENSIONS = {'txt', 'pdf', 'png', 'jpg', 'jpeg', 'gif'}

app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
app.config['MAX_CONTENT_LENGTH'] = MAX_FILE_SIZE

# Ensure upload folder exists
if not os.path.exists(UPLOAD_FOLDER):
    os.makedirs(UPLOAD_FOLDER)

# Check allowed file types
def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

# Simple HTML form
HTML = '''
<!doctype html>
<title>Upload File</title>
<h2>Upload File</h2>
<form method="post" enctype="multipart/form-data">
  <input type="file" name="file">
'''
```

```

    <input type="submit" value="Upload">
</form>
<p>{{ message }}</p>
'''

@app.route('/', methods=['GET', 'POST'])
def upload_file():
    message = ""

    if request.method == 'POST':
        if 'file' not in request.files:
            message = "No file part"
        else:
            file = request.files['file']

            if file.filename == '':
                message = "No selected file"

            elif not allowed_file(file.filename):
                message = "Invalid file type"

            else:
                filename = secure_filename(file.filename)
                file_path = os.path.join(app.config['UPLOAD_FOLDER'], filename)
                file.save(file_path)
                message = "File uploaded securely"

    return render_template_string(HTML, message=message)

if __name__ == '__main__':
    app.run(debug=True)

```

Output:

Upload File

No file chosen

File uploaded securely

Explanation:

This code builds a file upload system and checks for security risks before accepting any file. It prevents harmful uploads by restricting file types, limiting file size, and storing files safely.

Task 3: Real-Time Application: API Security Review

Prompt:

Generate a Python Flask API endpoint and then review and improve it by identifying security issues like missing authentication, unsafe HTTP methods, and no rate limiting, and fix them by adding token-based authentication, proper input validation, and rate limiting to create a secure API

Code:

```
from flask import Flask, request, jsonify
from functools import wraps
import time

app = Flask(__name__)

# In-memory token store (demo only)
tokens = {"valid_token": "user1"}

# Token authentication
def token_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        token = request.headers.get('Authorization')
        if not token or token not in tokens:
            return jsonify({"message": "Token is missing or invalid!"}), 401
        return f(*args, **kwargs)
    return decorated

# Simple rate limiter
rate_limit = {}

def rate_limiter(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        token = request.headers.get('Authorization')
        user = tokens.get(token)

        if user:
            current_time = time.time()
            if user in rate_limit and current_time - rate_limit[user] < 60:
                return jsonify({"message": "Rate limit exceeded!"}), 429
            rate_limit[user] = current_time

        return f(*args, **kwargs)
    return decorated

# Home route (fixes your 404 issue)
@app.route('/')
def home():
    return "API is running"

# Test API route
@app.route('/api', methods=['GET'])
def api():
```

```

    return jsonify({"message": "Secure API working"})

# Secure endpoint
@app.route('/data', methods=['POST'])
@token_required
@rate_limiter
def data():
    data = request.get_json()

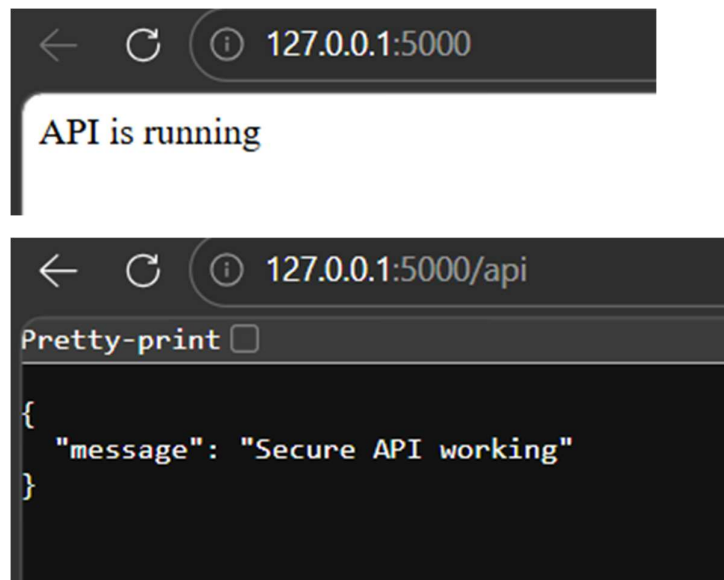
    if not data or 'value' not in data:
        return jsonify({"message": "Invalid input!"}), 400

    return jsonify({
        "message": "Data received!",
        "data": data['value']
    }), 200

if __name__ == '__main__':
    app.run(debug=True)

```

Output:



Explanation:

This code reviews an API endpoint to find security weaknesses like lack of authentication and validation. It then improves security by adding token-based access control, validating inputs, and limiting request rates to prevent misuse.

Task 4: Cross-Site Scripting (XSS) Check

Prompt:

Generate an HTML feedback form with JavaScript that displays user input on the page and then improve it by identifying XSS vulnerabilities such as directly rendering untrusted input, and fix them by escaping HTML entities and applying security measures like Content Security Policy (CSP) to prevent script injection

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Feedback Form</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 20px;
    }
    .feedback-form {
      max-width: 400px;
      margin: auto;
    }
    .feedback-form input, .feedback-form textarea {
      width: 100%;
      padding: 10px;
      margin: 10px 0;
    }
    .feedback-display {
      margin-top: 20px;
      padding: 10px;
      border: 1px solid #ccc;
    }
  </style>
</head>
<body>
  <div class="feedback-form">
    <h2>Feedback Form</h2>
    <input type="text" id="name" placeholder="Your Name">
    <textarea id="feedback" placeholder="Your Feedback"></textarea>
    <button onclick="submitFeedback()">Submit</button>
  </div>
  <div class="feedback-display" id="feedbackDisplay"></div>

  <script>
    function escapeHtml(unsafe) {
      return unsafe
        .replace(/&/g, "&amp;")
        .replace(/</g, "&lt;")
        .replace(/>/g, "&gt;")
        .replace(/"/g, "&quot;")
        .replace(/'/g, "&#039;");
    }

    function submitFeedback() {
      const name = document.getElementById('name').value;
      const feedback = document.getElementById('feedback').value;

      // Escape user input to prevent XSS
      const safeName = escapeHtml(name);
```

```

    const safeFeedback = escapeHtml(feedback);

    // Display the feedback safely
    const feedbackDisplay = document.getElementById('feedbackDisplay');
    feedbackDisplay.innerHTML = `<strong>${safeName}</strong> ${safeFeedback}`;
  }
</script>

<!-- Content Security Policy to prevent script injection -->
<meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self'; style-src 'self';">
</body>
</html>

```

Output:

Explanation:

This code checks if user input is safely handled before displaying it in the browser. It prevents XSS attacks by escaping harmful scripts and enforcing security policies like CSP.

Task 5: SQL Injection Prevention

Prompt:

Generate a Python script that uses SQLite to fetch user details from a database and then improve it by identifying SQL injection vulnerabilities such as string concatenation in queries and fixing them using parameterized queries (prepared statements) to make it secure

Code:

```

import sqlite3

# Create and initialize database (only runs once safely)
def setup_database():
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()

    cursor.execute('''
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY,
            name TEXT
        )
    ''')

```

```

# Insert sample data (only if empty)
cursor.execute("SELECT COUNT(*) FROM users")
if cursor.fetchone()[0] == 0:
    cursor.execute("INSERT INTO users (name) VALUES ('alice')")
    cursor.execute("INSERT INTO users (name) VALUES ('bob')")
    cursor.execute("INSERT INTO users (name) VALUES ('likitha')")
    conn.commit()

conn.close()

# Insecure function (SQL Injection vulnerable)
def get_user_insecure(user_id):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()

    query = f"SELECT * FROM users WHERE id = {user_id}" # ✗ vulnerable
    cursor.execute(query)
    result = cursor.fetchall()

    conn.close()
    return result

# Secure function (using parameterized query)
def get_user_secure(user_id):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()

    query = "SELECT * FROM users WHERE id = ?" # ✓ safe
    cursor.execute(query, (user_id,))
    result = cursor.fetchall()

    conn.close()
    return result

# Main program
if __name__ == "__main__":
    setup_database()

    user_id = input("Enter user ID: ")

    print("\nFetching user details (insecure):")
    try:
        print(get_user_insecure(user_id))
    except Exception as e:
        print("Error:", e)

    print("\nFetching user details (secure):")
    try:
        print(get_user_secure(user_id))
    except Exception as e:
        print("Error:", e)

```


Output:

```
PS C:\Users\lilac\New folder\fashion-store\server> & C:\Users\lilac\New folder\fashion-store\server\ai5.py
Enter user ID: 1

Fetching user details (insecure):
[(1, 'alice')]

Fetching user details (secure):
[(1, 'alice')]
PS C:\Users\lilac\New folder\fashion-store\server> █
```

Explanation:

This code checks how user input is used in database queries and identifies risks of SQL injection. It prevents attacks by using parameterized queries instead of directly inserting user input into SQL statements.